such hidden problems difficult to spot with ML testing, since we typically depend on a point measure, namely 'accuracy', on a held-out test data set as the success/failure threshold.

One way of overcoming some of these issues is by using metamorphic testing. Instead of testing for a specific output (given a specific input), metamorphic testing looks at patterns/relationships between inputs and their corresponding outputs. Consider the example sentence used above for sentiment analysis model. If we replace the word 'movie' with its synonym 'film', the output response should not change. Such a metamorphic test pattern does not need to know what the correct output is, so we don't need a test oracle.

Metamorphic testing also allows us to weed out some of the bias/unfairness related problems that our built model may have incorporated unintentionally. Consider the case of a hate speech classifier model. Given that hate speech is often directed at gay people, it is possible that the model has learnt an association between the word 'gay' and the hate speech label. While a sentence such as "I hate all gay people" is a true example of hate speech, the model can misclassify sentences such as "The child was gay and playful when we met," as hate speech because of the wrongly identified word association. Metamorphic testing for synonym patterns can reveal this problem quite easily since the modified sentence, "The child was happy and playful when we met," would be classified as non-hate speech. The flip in the predicted label when a word is replaced by a synonym typically indicates a lack of robustness for the learnt model. This kind of metamorphic testing allows us to test for many instances even when we don't have a test oracle readily available. Just as we check for label equivalence with synonyms for a sentiment analysis classification problem, we can also check for label flip when we replace a word with its antonym. Can you think of other patterns that would allow you to extend your test inputs without having to explicitly label them?

If you have been reading this article carefully, you would have caught a flaw in my argument. Consider my claim that when we replace a word with its antonym, the model should flip its predicted label. While this statement probably holds true for the specific task of sentiment analysis that we have talked about, does this claim hold true for other ML/NLP tasks as well? Not really.

For instance, consider a relation classification problem for which we are looking for different relation types such as 'leader of', 'located in', 'employee of', 'the property of', etc. Consider the sentence: "Baldwins is the best school in Bangalore." The relation classification model correctly labels this as 'location-relation'. Now let us assume that we flip the word 'best' with its antonym 'worst', so that the new input sentence is, "Baldwins is the worst school in Bangalore." While the meaning of the sentence has changed, it has no impact on the 'location-relation' we are interested in. Hence, such adversarial test input generation is task-specific, and one needs to be careful when applying these rules to generate adversarial text to consider the task on hand.

The above example brings us to the question of what properties we need to test for in an ML/NLP application. We have seen that robustness to adversarial examples is an important property for the ML applications. What are some of the other desirable properties? We briefly mentioned about fairness/immunity to algorithmic bias as a desirable property. Algorithmic bias is related to undesirable/unfair outcomes created by an ML model discriminating against specific individuals/groups based on certain protected attributes such as race, gender, caste, country, etc.

Making sure that your ML model is not exhibiting algorithmic bias is critical in any real-world deployment where this model can help make decisions that impact human life. Examples of such situations include loan processing, credit card applications, granting parole, etc.

While everyone agrees that ML models should be free of algorithmic bias, there is much confusion over what constitutes unfairness and how to mitigate it. There are multiple ways your model can exhibit algorithmic bias. Consider the case that your input data itself contains samples which can lead to algorithmic bias. For instance, loan applications can have input data about the applicants. Given your training data distribution, it is possible that many of the people whose loans were rejected belonged to one specific race or gender. This bias in your data can lead to discrimination against one specific race or gender. This problem can be mitigated by making sure that none of the protected attributes such as race, gender or caste are used as a feature. Even when none of the protected attributes are directly used as a feature, it is possible for algorithmic bias to creep in.

Let us look at an example where your classifier model uses the ZIP code as a feature. If the ZIP code maps to a locality that predominantly houses one specific racial community or a specific caste, then the ZIP code can become a proxy attribute for a protected attribute in many cases. This will lead to the model developing implicit algorithmic bias. While we have so far talked about the issues with algorithmic bias, how does one explicitly test whether your model has it? We will discuss this in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Wishing all our readers happy coding until next month! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.